

HOME HEALTH EQUIPMENT RENTAL DATABASE

CARENT MEDICAL SUPPLY LLC

HMI 7550: Data Systems in Healthcare — Group Project Written Report

Davis, Jordan

jdavi376@students.kennesaw.edu

Kacha, Gopi

gkacha@students.kennesaw.edu

Jenkins, Shelese

sjenki50@students.kennesaw.edu

Clark, Bridget

bweiss3@students.kennesaw.edu

Kennesaw State University

Spring Semester 2026

Abstract

This report presents the design and implementation of a Transaction Processing System (TPS) relational database for CareRent Medical Supply LLC, a fictitious durable medical equipment rental company in Atlanta, Georgia. The database centralizes patient records, equipment inventory, rental agreements, insurance billing, and delivery schedules to improve operational efficiency and data management. The project includes Entity-Relationship (ER) modeling with five core entities and SQL-based table creation. Additionally, the report explains the project rationale, database design methodology, SQL implementation process, key findings, and overall outcomes. Group reflections, lessons learned, and recommendations for future improvements are also discussed throughout the report.

Introduction

The home health and durable medical equipment (DME) rental industry plays a critical role in the continuum of care, providing patients recovering from surgery or managing chronic conditions with essential equipment such as wheelchairs, hospital beds, and oxygen concentrators. Despite the importance of this sector, many DME companies continue to operate using fragmented, manual processes—primarily spreadsheets and paper records—that are ill-suited to handle growing operational complexity.

Without a centralized information system, these companies face persistent challenges: billing errors due to inconsistent data entry, difficulty tracking equipment location and availability, inefficient scheduling of delivery technicians, and growing compliance risks associated with Medicare, Medicaid, and private insurance billing requirements. These inefficiencies not only inflate operational costs but also degrade the quality and reliability of patient care.

This project addresses those challenges by designing and implementing a Transaction Processing System (TPS) database for CareRent Medical Supply LLC, a fictional mid-sized DME rental company. A TPS is a type of information system that collects, stores, modifies, and retrieves routine daily transactional data for an organization (Stair & Reynolds, 2020). For CareRent, the TPS will automate rental tracking, streamline insurance claims processing, and improve coordination across departments.

The objectives of this project are to: (1) develop an Entity-Relationship (ER) model representing all core operational entities and their relationships and (2) construct a relational database using SQL, including base tables and supporting tables. This report details each phase of the project, reflects the group's experience, and concludes with future recommendations for CareRent.

Database and Data Description

Database Purpose and Scope

The CareRent TPS database is designed to centralize all core operational data and support the processing of daily transactions related to patient rentals, equipment inventory, insurance billing, and technician scheduling. The database serves multiple user roles, including billing staff, inventory coordinators, delivery drivers, and operations managers, each interacting with the system through their respective functional areas.

Data Description

Since no existing CareRent dataset was available, all data used in this project is fictitious and was generated to reflect realistic DME rental business scenarios. The data generation strategy included:

- Excel functions (RAND(), RANDBETWEEN()) to generate patient IDs, rental dates, equipment serial numbers, and billing amounts.
- Manual entry for constructing business scenarios such as rental agreements, maintenance logs, and delivery schedules.

The generated dataset includes records across all five core entities and their supporting tables, structured to reflect realistic operational volume for a mid-sized DME company. All data was imported into the MySQL database environment for testing and demonstration purposes.

Database Modeling

Entities and Attributes

The CareRent database is structured around five core entities and one associative entity. The following table summarizes each entity, its primary key, and its key attributes:

Table 1. CareRent Entities, Primary Keys, and Key Attributes

Entity	Primary Key	Key Attributes
Patient	PatientID	FirstName, LastName, Phone, AddressStreetName, AddressZipCode, InsuranceID (FK)
Equipment	EquipmentID	EquipmentID, EquipmentName, EquipmentCategory, EquipmentDailyRate
Rental Agreement	RentalAgreementID	StartDate, EndDate, RentalReturnStatus, PatientID (FK), EquipmentID (FK)
Delivery	DeliveryID	DeliveryDate, DeliveryStatus, RentalAgreementID, DriverID
Insurance	InsuranceID	InsuranceName, PlanType, MaxCoverageAmount, PatientID
Driver	DriverID	DriverLastName, DriverFirstName, DriverDeliveryRegion, DriverPhoneNumber

Entity Descriptions

Patient: The Patient entity stores all demographic and insurance-related information for individuals who rent equipment from CareRent. Each patient is assigned a unique PatientID and is linked to an Insurance/Billing Plan through a foreign key, allowing the system to track coverage and claims at the individual level.

Equipment: The Equipment entity maintains an inventory of all DME assets owned by CareRent. Each item is identified by a unique EquipmentID. Key operational attributes include Category (e.g., mobility, respiratory) and DailyRate—enabling accurate billing and availability tracking.

Rental Agreement: The Rental Agreement is the central associative entity of the database. It resolves the many-to-many relationship between Patient and Equipment Item while also linking to the Delivery Driver who is responsible for transporting the equipment. It records the start and end dates of each rental, the negotiated rental rate, current return status, and delivery type (e.g., pickup, drop-off, courier).

Delivery Driver: The Delivery Driver entity tracks the certified service technicians responsible for delivering and retrieving equipment. Each driver is assigned a region and is linked to rental agreements through a one-to-many relationship, meaning a single driver can be responsible for multiple deliveries.

Insurance/Billing Plan: The Insurance/Billing Plan entity stores details about the coverage plans associated with patients. This includes the insurance provider name, plan type (e.g., Medicare, Medicaid, Private), coverage amount, billing cycle date, and current claim status. Multiple patients may share the same insurance plan, creating a many-to-one relationship between Patient and Insurance/Billing Plan.

Entity Relationships

The relationships among entities are defined as follows:

Table 2. CareRent Entity Relationships

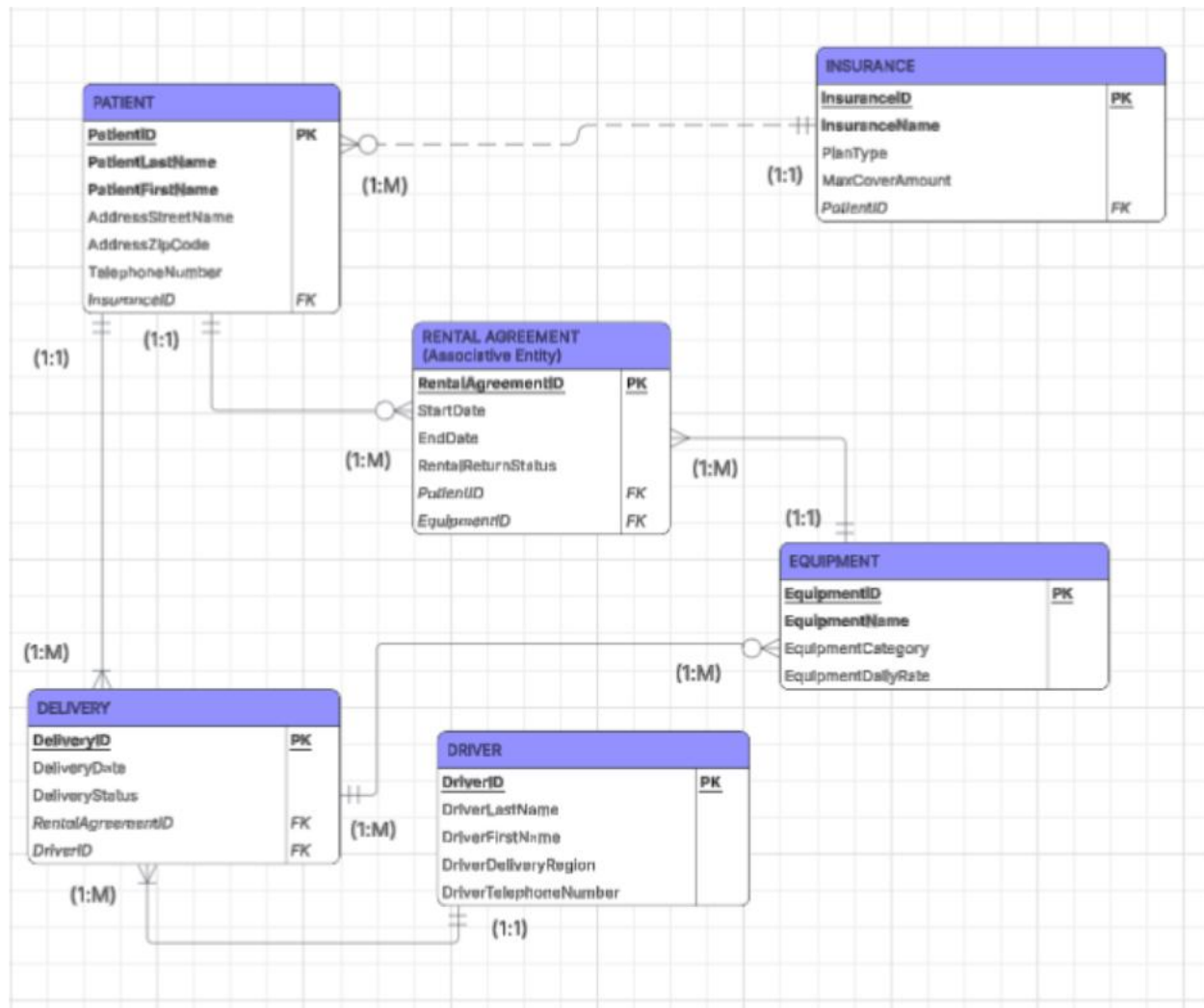
Relationship	Type	Description
Patient — Rental Agreement	One-to-Many	One patient can have multiple rental agreements over time.
Rental Agreement — Patient	One-to-One	One Rental Agreement can be assigned to one patient.
Equipment — Rental Agreement	One-to-Many	One equipment item can appear across many rental agreements.
Delivery — Driver	One-to-One	Each delivery can be assigned to one driver.
Driver — Delivery	One-to-Many	One driver can be assigned to many deliveries.
Delivery — Equipment	One-to-Many	One delivery can include many equipment items. Many equipment items can be in a single delivery.
Insurance — Patient	One-to-Many	One Insurance company can cover many patients.
Patient — Insurance	One-to-One	One patient has one insurance.
Patient — Delivery	One-to-Many	One patient can have many deliveries.
Delivery — Patient	One-to-One	Each delivery is delivered to one patient.

The Rental Agreement entity serves as the hub of the relational model. By acting as an associative entity, it resolves the inherent many-to-many relationship between Patient and Equipment Item—a common structural challenge in healthcare operational databases. Foreign keys (PatientID, EquipmentID) embedded in the Rental Agreement enforce referential integrity and ensure that no orphaned records exist.

Entity-Relationship Diagram

The Entity-Relationship (ER) diagram was developed using Lucidchart and reflects all five core entities, their attributes, primary keys, foreign keys, and the cardinality of each relationship. The diagram illustrates the central role of the Rental Agreement as an associative entity linking Patient, Equipment Item, and Delivery Driver. Insurance is positioned as a reference entity accessed by the Patient entity. The Delivery entity is represented as a supporting table connected to both the Equipment and Driver entity.

The ER model uses Crow's Foot notation to represent cardinality, with mandatory and optional participation indicators on each relationship line. The model was iteratively refined throughout the project to accommodate additional business rules identified during data generation and SQL development.



[Figure 1: CareRent Medical Supply LLC Entity-Relationship Diagram — See Appendix]

Normalization

Normalization was not needed for this project due to fictitious data being created and used.

Writing SQL Statements

Overview of SQL Usage

Structured Query Language (SQL) was used throughout this project to define the database schema, insert data, query records, and enforce data integrity. MySQL was selected as the database management system (DBMS) due to its widespread use in healthcare information systems and its robust support for relational data modeling.

The SQL development for this project was divided into four primary categories: Data Definition Language (DDL) commands to create and structure tables, Data Manipulation Language (DML) commands to insert and manage data, referential integrity enforcement through primary and foreign key constraints, and SELECT queries for data retrieval and reporting.

DDL — Table Creation

CREATE TABLE statements were used to define all six tables in the CareRent database schema. For each table, column names, data types, primary keys, and foreign keys were explicitly defined. For example, the Patient table was created with PatientID as the auto-incrementing primary key, and InsurancePlanID was defined as a foreign key referencing the Insurance/Billing Plan table. This approach enforces referential integrity at the database level, preventing orphaned records.

The Rental Agreement table required particular attention because it contains three foreign keys (PatientID, EquipmentID), making it the most relationally complex table in the schema. The order of table creation was carefully managed so that referenced tables were always created before the tables referencing them—a common requirement when working with foreign key constraints in MySQL.

DML — Data Insertion

INSERT INTO statements were used to populate all tables with fictitious data generated in Excel and manually constructed business scenarios. Batch INSERT statements were used where possible to improve efficiency. Data validation was performed after insertion using SELECT queries to verify that records appeared as expected and that all foreign key references resolved correctly.

Supporting Table Creation

Three supporting tables—Rental Line Items, Maintenance Log, and Delivery Schedule—were created using SQL as part of the new table creation phase of the project. These tables were not part of the initial ER model design but were identified as necessary additions during the SQL development phase to capture granular operational data that could not be represented adequately in the core entity tables.

The Delivery Schedule table, for example, was created to associate specific delivery events with equipment items, rental agreements, and drivers, and includes fields for DeliveryDate, DeliveryTime, and Address. This table enables CareRent managers to view a complete delivery calendar and assign or reassign drivers dynamically.

SELECT Queries for Reporting

Multiple SELECT queries were developed to generate reports that support management decision-making. These queries used JOIN operations to combine data across multiple tables, WHERE clauses to filter records by status or date range, and aggregate functions (COUNT, SUM, AVG) to calculate equipment utilization rates, rental revenue totals, and driver workload distributions. These query results were also used as the basis for the data visualizations described in the following section.

Detailed SQL code for all CREATE TABLE statements, INSERT statements, and SELECT queries is provided in the Appendix.

Narrative Interpretation and Conclusion

Project Experience and Challenges

This project was a valuable and challenging learning experience that required our group to apply both technical and collaborative skills. Designing a relational database for a healthcare operational environment introduced challenges beyond typical classroom exercises, particularly in the areas of referential integrity, normalization, and relational database design.

One of the most difficult parts of the project was inserting the data into MySQL. During this process, we encountered several MySQL error codes and constraint violations while loading data into the tables. Ensuring that the data matched the correct column data types and that foreign key values referenced existing primary key records required careful attention to detail. The group had to troubleshoot issues related to table relationships, data formatting, and the order in which records were inserted into the database.

Another significant challenge involved creating realistic fictitious data that accurately represented the operations of a durable medical equipment (DME) rental company. The data needed to remain consistent across all tables, particularly between the Patient, Rental Agreement, Equipment, Delivery, and Driver tables. To maintain referential integrity, we had to ensure that related records were properly linked before inserting dependent records into child tables.

From a technical perspective, writing multi-table JOIN queries for reporting purposes was one of the most advanced aspects of the project. Combining data across several related tables required careful use of aliases, join conditions, and filtering statements to ensure accurate query results. The group also used aggregate functions and reporting queries to analyze rental activity, delivery information, and equipment usage.

Overall, the group collaborated effectively throughout the project. Each member contributed to their assigned responsibilities, and communication was maintained through regular discussions and progress updates. While coordinating schedules occasionally presented challenges, the group remained flexible and worked together successfully to complete the project.

Limitations

Several limitations were encountered during the course of this project:

- **Fictitious Data:** Because all data was synthetically generated, the database does not reflect the full complexity and variability of a real-world DME operation. Edge cases, irregular billing scenarios, and multi-insurance patients were simplified or excluded.
- **Scope Constraints:** The database does not currently include a full inventory tracking system with real-time status updates, a patient portal, or integration with external insurance eligibility systems—features that would be essential in a production environment.
- **Security and Compliance:** The current implementation does not address HIPAA compliance requirements, role-based access control, or data encryption, which are mandatory considerations for any real healthcare data system.
- **Scalability:** The database was designed for a mid-sized DME company. Scaling the system to support a larger organization or multi-location operations would require architectural changes, including table partitioning, indexing strategies, and potentially a distributed database design.

Conclusion and Future Recommendations

This project successfully delivered a fully normalized, SQL-based Transaction Processing System database for CareRent Medical Supply LLC. The database centralizes all core operational data, enforces referential integrity, and provides the foundation for accurate billing, efficient scheduling, and reliable equipment tracking. The data visualizations developed as part of the project provide CareRent management with actionable insights into equipment utilization, rental revenue trends, and technician workloads.

Looking ahead, we recommend the following enhancements to strengthen and extend the CareRent database system:

- **HIPAA Compliance Implementation:** Incorporate role-based access control, audit logging, and data encryption to meet federal healthcare privacy requirements before any real patient data is processed.
- **Real-Time Equipment Tracking:** Integrate IoT-enabled asset tracking to provide live equipment status updates and reduce the risk of lost or unaccounted inventory.
- **Electronic Insurance Verification:** Connect the billing module to real-time insurance eligibility APIs to reduce claim denials and accelerate reimbursement cycles.
- **Business Intelligence Dashboard:** Expand the current visualizations into a live dashboard connected directly to the MySQL database, enabling management to monitor KPIs in real time.

- **Mobile Access for Drivers:** Develop a mobile interface for delivery drivers to view and update their delivery schedules, confirm deliveries, and log maintenance issues from the field.

In conclusion, the CareRent TPS database project demonstrated the practical application of relational database design principles to a real-world healthcare operational context. By building a structured, normalized, and query-ready system, CareRent is positioned to eliminate the inefficiencies of its current manual processes and build a scalable data infrastructure to support future growth.

References

- Centers for Medicare & Medicaid Services. (2023). *Durable medical equipment (DME) center*. U.S. Department of Health and Human Services. <https://www.cms.gov/center/dme>
- Coronel, C., & Morris, S. (2021). *Database systems: Design, implementation, and management* (14th ed.). Cengage Learning.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson Education.
- OpenAI. (2026). *ChatGPT (May 2026 version) [Large language model]*. <https://chat.openai.com>
- Stair, R., & Reynolds, G. (2020). *Principles of information systems* (14th ed.). Cengage Learning.

Appendix

Appendix A: Entity-Relationship Diagram

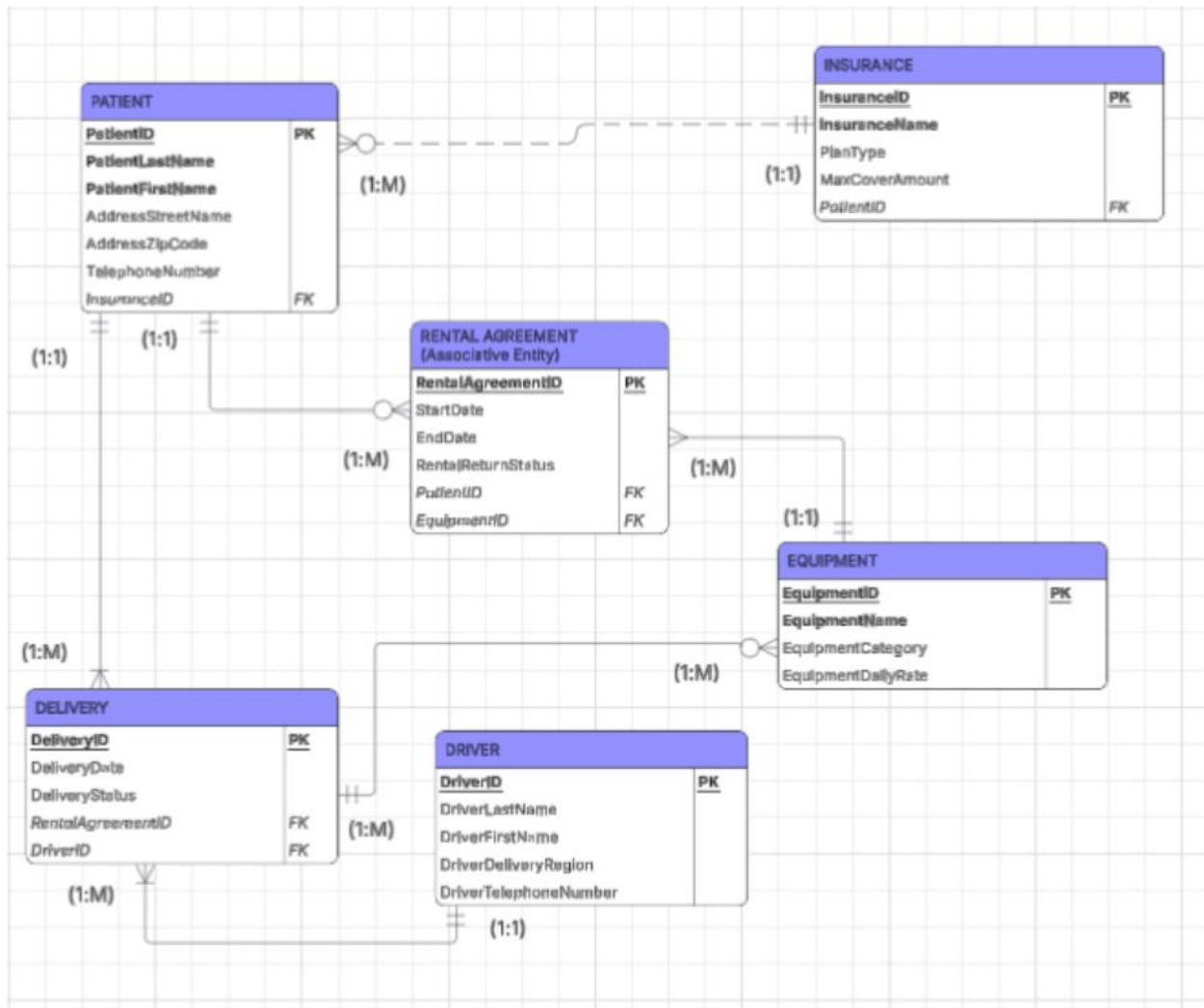


Figure 1. CareRent Medical Supply LLC Entity-Relationship Diagram

Appendix B: NF Diagram

N/A — Normalization was not required for this project due to the use of fictitious data.

Appendix C: SQL CREATE TABLE Statements

```
CREATE TABLE Patient (
    PatientID INT PRIMARY KEY,
    PatientLastName VARCHAR(50),
    PatientFirstName VARCHAR(50),
    AddressStreetName VARCHAR(100),
    AddressZipCode VARCHAR(10),
    TelephoneNumber VARCHAR(20),
    InsuranceID VARCHAR(20));
```

```
CREATE TABLE Insurance (
    InsuranceID VARCHAR(20) PRIMARY KEY,
```

```
InsuranceName VARCHAR(100),
PlanType VARCHAR(50),
MaxCoverAmount DECIMAL(10,2),
PatientID INT);
```

```
CREATE TABLE Equipment (
    EquipmentID INT PRIMARY KEY,
    EquipmentName VARCHAR(100),
    EquipmentCategory VARCHAR(50),
    EquipmentDailyRate DECIMAL(10,2));
```

```
CREATE TABLE Driver (
    DriverID INT PRIMARY KEY,
    DriverLastName VARCHAR(50),
    DriverFirstName VARCHAR(50),
    DriverDeliveryRegion VARCHAR(50),
    DriverTelephoneNumber VARCHAR(20));
```

```
CREATE TABLE RentalAgreement (
    RentalAgreementID VARCHAR(20) PRIMARY KEY,
    StartDate DATE,
    EndDate DATE,
    RentalReturnStatus VARCHAR(50),
    PatientID INT,
    EquipmentID INT,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    FOREIGN KEY (EquipmentID) REFERENCES Equipment(EquipmentID));
```

```
CREATE TABLE Delivery (
    DeliveryID INT PRIMARY KEY,
    DeliveryDate DATE,
    DeliveryStatus VARCHAR(50),
    RentalAgreementID VARCHAR(20),
    DriverID INT,
    FOREIGN KEY (RentalAgreementID) REFERENCES RentalAgreement(RentalAgreementID),
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID));
```

Appendix D: Sample SELECT Queries

View all patients:

```
SELECT * FROM Patient;
```

Patients with their insurance details:

```
SELECT p.PatientID, p.PatientFirstName, p.PatientLastName,
       i.InsuranceName, i.PlanType, i.MaxCoverAmount
FROM Patient p
JOIN Insurance i ON p.InsuranceID = i.InsuranceID;
```

List equipment with daily rental cost:

```
SELECT EquipmentName, EquipmentCategory, EquipmentDailyRate FROM Equipment;
```

Show all rental agreements with patient and equipment information:

```
SELECT r.RentalAgreementID, r.StartDate, r.EndDate, r.RentalReturnStatus,
       p.PatientFirstName, p.PatientLastName, e.EquipmentName, e.EquipmentCategory
FROM RentalAgreement r
JOIN Patient p ON r.PatientID = p.PatientID
JOIN Equipment e ON r.EquipmentID = e.EquipmentID;
```

Find active rentals:

```
SELECT * FROM RentalAgreement WHERE RentalReturnStatus <> 'Returned';
```

Show delivery records with driver and rental information:

```
SELECT d.DeliveryID, d.DeliveryDate, d.DeliveryStatus,
       dr.DriverFirstName, dr.DriverLastName, r.RentalAgreementID
FROM Delivery d
JOIN Driver dr ON d.DriverID = dr.DriverID
JOIN RentalAgreement r ON d.RentalAgreementID = r.RentalAgreementID;
```

Count deliveries per driver:

```
SELECT dr.DriverFirstName, dr.DriverLastName,
       COUNT(d.DeliveryID) AS TotalDeliveries
FROM Driver dr
LEFT JOIN Delivery d ON dr.DriverID = d.DriverID
GROUP BY dr.DriverFirstName, dr.DriverLastName;
```

Find patients who rented equipment:

```
SELECT DISTINCT p.PatientID, p.PatientFirstName, p.PatientLastName
FROM Patient p
JOIN RentalAgreement r ON p.PatientID = r.PatientID;
```

Show rental duration:

```
SELECT RentalAgreementID, StartDate, EndDate,
       DATEDIFF(EndDate, StartDate) AS RentalDays
FROM RentalAgreement;
```

Appendix E: AI Review of ER Model

After using ChatGPT to review the CareRent Medical Supply LLC ER model, the criticisms of the ER model helped us better understand where the design could be improved. The feedback showed that some relationships and foreign keys were repetitive and made the database more complicated than it needed to be. We also learned that cardinality and participation symbols need to clearly reflect how the system would work in a real-world situation. Another important point was keeping attribute names consistent and making sure relationships between entities are simple and logical. Overall, the critiques helped us see common mistakes in database design and showed how making small changes can greatly improve the organization and efficiency of an ER model.